ЛАБОРАТОРНАЯ РАБОТА.

АВТОМАТИЗАЦИЯ РАЗВЁРТЫВАНИЯ ПРОЕКТА

С ПОМОЩЬЮ GITHUB ACTIONS (CI/CD)

1. ЦЕЛЬ И ЗАДАЧИ ПРАКТИЧЕСКОЙ РАБОТЫ

Цель: получение практических навыков настройки автоматического развёртывания (CI/CD) веб-проекта на хостинге с использованием сервиса GitHub Actions.

Для достижения поставленной цели требуется решить следующие задачи:

1. Создать репозиторий проекта на сервисе GitHub с файлами статического веб-сайта.
2. Опубликовать сайт на хостинге (GitHub Pages, Vercel или Netlify).
3. Разработать workflow-файл GitHub Actions, автоматически запускающий сборку и развёртывание проекта при каждом push в репозиторий.
4. Убедиться, что изменение файлов в репозитории приводит к автоматическому обновлению сайта на хостинге без ручных действий.
5. Реализовать индивидуальный проект с применением настроенного CI/CD-пайплайна.

Структура методических рекомендаций

В разделе 1 представлены цели и задачи практического задания.

В разделе 2 показаны критерии оценивания. Внимательно изучите этот раздел.

В разделе 3 рассмотрены теоретические основы CI/CD и устройство сервиса GitHub Actions.

В разделе 4 представлена последовательность выполнения упражнений по настройке автоматического развёртывания.

В конце методических указаний представлены список литературы и пример выполнения отчёта в Приложениях.

Общие рекомендации

GitHub Actions — это встроенный в GitHub сервис непрерывной интеграции и доставки (CI/CD), позволяющий автоматически выполнять сценарии (workflow) в ответ на события в репозитории, например на команду git push. Вместо того чтобы вручную загружать изменённые файлы на хостинг, разработчик один раз описывает сценарий действий в

YAML-файле, и далее платформа сама пересобирает и публикует проект при каждом изменении.

Перед выполнением задания подготовьте простой статический сайт (файлы index.html и style.css) либо используйте проект из предыдущей лабораторной работы. Дополнительно потребуется аккаунт на GitHub и (по желанию) аккаунт на Vercel или Netlify для сравнения различных способов автодеплоя.

Внимательно прочитайте методические указания и выполните упражнения по порядку. Интерфейс GitHub может обновляться — в указаниях описано расположение элементов по состоянию на 2026 год.

Если справиться с заданием не удаётся, покажите в отчёте на скриншотах, на каком этапе возникла проблема, и дайте пояснение в выводе.

После выполнения упражнений переходите к индивидуальному проекту: добавьте собственную страницу или доработайте workflow (например, настройте деплой в несколько окружений, добавьте шаг тестирования или линтинга перед публикацией).

Последовательность выполнения задания

1. Создать (или использовать существующий) репозиторий на GitHub с файлами статического сайта.
2. Включить публикацию сайта одним из способов: GitHub Pages, Vercel или Netlify.
3. Создать в репозитории файл .github/workflows/deploy.yml, описывающий автоматическую сборку и деплой.
4. Зафиксировать (commit) и отправить (push) изменения, проверить успешное выполнение workflow во вкладке Actions.
5. Внести изменение в любой файл проекта, отправить push и убедиться, что сайт обновился автоматически.
6. Выполнить индивидуальный проект, предварительно согласовав тему с преподавателем.
7. Оформить отчёт в соответствии с требованиями и проверить критерии оценивания (раздел 2).

Структура отчёта по практической работе

Цель работы.

1. Ссылка на репозиторий проекта на GitHub со статическим сайтом.
2. Скриншот содержимого файла workflow (.github/workflows/deploy.yml).
3. Скриншот вкладки Actions с успешным выполнением workflow (зелёная галочка).
4. Скриншоты сайта на хостинге до и после изменения файла в репозитории.

5. Ссылка на опубликованный сайт (GitHub Pages / Vercel / Netlify).
6. Текстовые комментарии и скриншоты индивидуального проекта.
7. Вывод в формате эссе: цель работы, полученные знания и навыки, возникшие проблемы и пути их решения.
8. Список использованных источников.

2. КРИТЕРИИ ОЦЕНИВАНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

Оценивание практической работы выполняется по следующим критериям. За выполнение работы выставляются баллы, которые переводятся в оценку в случае дифференцированного зачёта или зачёт/не зачёт в случае зачёта без оценки.

Таблица 2.1 — Критерии оценивания практической работы

№	Критерий	Балл
1	Цели и задачи работы сформулированы	Да — 1 балл / Нет — 0 баллов
2	Дана ссылка на репозиторий проекта	Да — 1 балл / Нет — 0 баллов
3	Репозиторий содержит статический сайт (не менее 2 файлов: .html, .css)	Да — 1 балл / Нет — 0 баллов
4	Создан и корректно настроен workflow-файл GitHub Actions (.github/workflows)	Да — 3 балла / Нет — 0 баллов
5	Продемонстрирован успешный автоматический запуск workflow при push (вкладка Actions)	Да — 2 балла / Нет — 0 баллов
6	Показано автоматическое обновление сайта на хостинге после изменения файла в репозитории (до/после)	Да — 3 балла / Нет — 0 баллов
7	Приведены скриншоты, подтверждающие выполнение задания (не менее 5-6 скриншотов)	Да — 2 балла / Нет — 0 баллов
8	Присутствует текстовое пояснение к скриншотам	Да — 1 балл / Нет — 0 баллов
9	Выполнен индивидуальный проект с использованием CI/CD	Да — 3 балла / Нет — 0 баллов
10	В выводе сформулированы решённые задачи и назначение CI/CD	Да — 2 балла / Нет — 0 баллов
11	Список использованных источников	Да — 1 балл / Нет — 0 баллов
	Максимально возможный балл	**20**

Таблица 2.2 — Перевод баллов в оценку

Баллы	Оценка	Зачёт/не зачёт
Менее 9 баллов	неудовлетворительно	Не зачёт
9-13 баллов	удовлетворительно	Зачёт

Баллы	Оценка	Зачёт/не зачёт
14-17	хорошо	
18-20	отлично	

3. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ CI/CD И GITHUB ACTIONS

CI/CD (Continuous Integration / Continuous Delivery, непрерывная интеграция и непрерывная доставка) — это подход к разработке, при котором изменения кода автоматически собираются, проверяются и доставляются пользователю без ручного вмешательства на каждом этапе.

GitHub Actions — встроенный в GitHub механизм автоматизации, позволяющий описывать сценарии (workflow) в файлах формата YAML, размещённых в каталоге .github/workflows репозитория. Основные понятия сервиса:

- Workflow — автоматизированный процесс, состоящий из одного или нескольких job’ов; описывается YAML-файлом.
- Event (событие) — триггер запуска workflow, например push, pull_request или ручной запуск workflow_dispatch.
- Job (задание) — набор шагов, выполняемых на одном исполнителе (runner).
- Step (шаг) — отдельная команда или готовое действие (action) внутри job.
- Action — переиспользуемый блок кода из GitHub Marketplace, например actions/checkout или actions/setup-node.
- Runner — виртуальная машина (например, ubuntu-latest), на которой выполняется workflow.
- Secrets — защищённые переменные окружения (токены, ключи API), хранящиеся в настройках репозитория.

Способы автоматического развёртывания сайта

Существует два основных подхода к автообновлению сайта при изменениях в репозитории:

Хостинг	Механизм автодеплоя	Особенности
GitHub Pages	Явный workflow GitHub Actions (actions/deploy-pages) либо встроенная сборка из ветки	Бесплатно, требует настройки прав workflow в Settings → Pages
Vercel	Нативная интеграция с GitHub — деплой запускается автоматически без YAML-файла	Удобно для фронтенд-фреймворков, есть preview-деплои для веток
Netlify	Аналогично Vercel: подключение репозитория и автосборка при push	Поддержка форм, функций, собственный CI

В упражнениях 3-4 рассмотрен вариант с явным workflow-файлом GitHub Actions (максимально гибкий и переносимый способ), в упражнении 5 — вариант нативной интеграции хостинга с GitHub.

4. ВЫПОЛНЕНИЕ УПРАЖНЕНИЙ

Упражнение 1. Подготовка репозитория со статическим сайтом

1. Создайте на GitHub новый репозиторий, например site-cicd. В описание внесите: «Автодеплой через GitHub Actions».

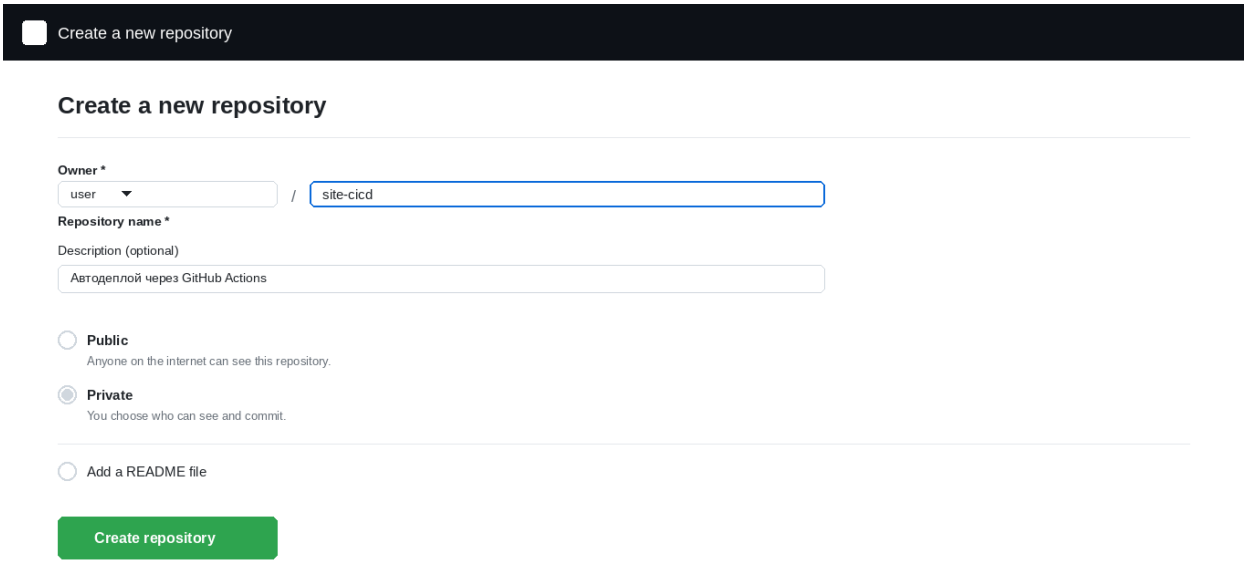


Рисунок 4.1 — Создание репозитория

2. Добавьте в репозиторий минимум два файла статического сайта: index.html и style.css. Файлы можно создать прямо в веб-интерфейсе GitHub (Add file → Create new file) либо загрузить с компьютера.

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <title>Мой сайт</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Привет, GitHub Actions!</h1>
</body>
</html>
```

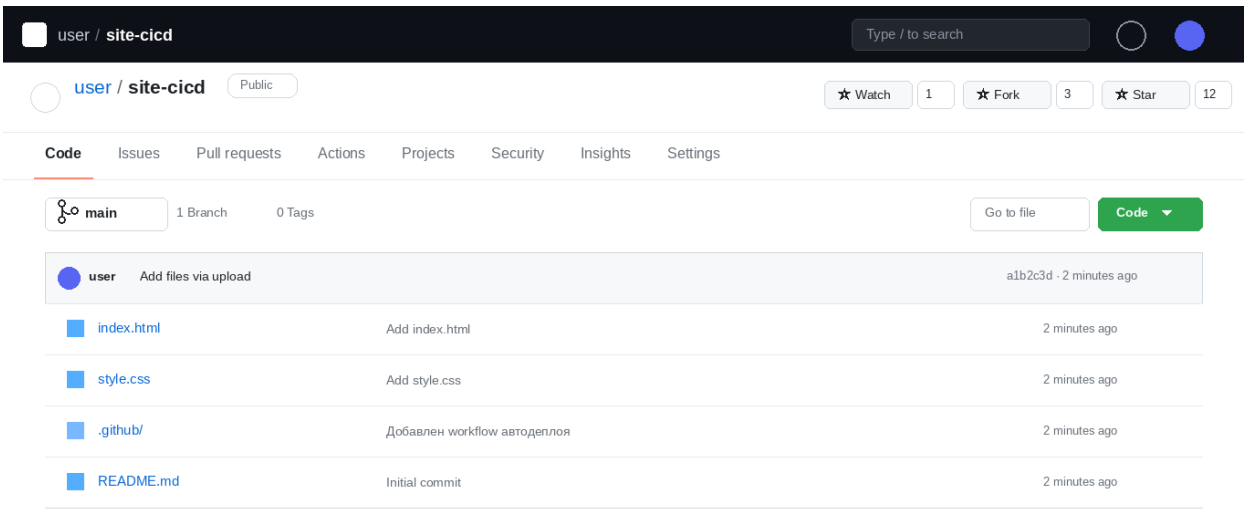


Рисунок 4.2 — Файлы статического сайта в репозитории

Упражнение 2. Публикация сайта на GitHub Pages (базовый способ)

1. Перейдите в Settings → Pages выбранного репозитория.

2. В разделе Build and deployment выберите источник Deploy from a branch, укажите ветку main и папку / (root).

3. Сохраните настройки и дождитесь появления ссылки на опубликованный сайт вида https://<username>.github.io/site-cicd/.

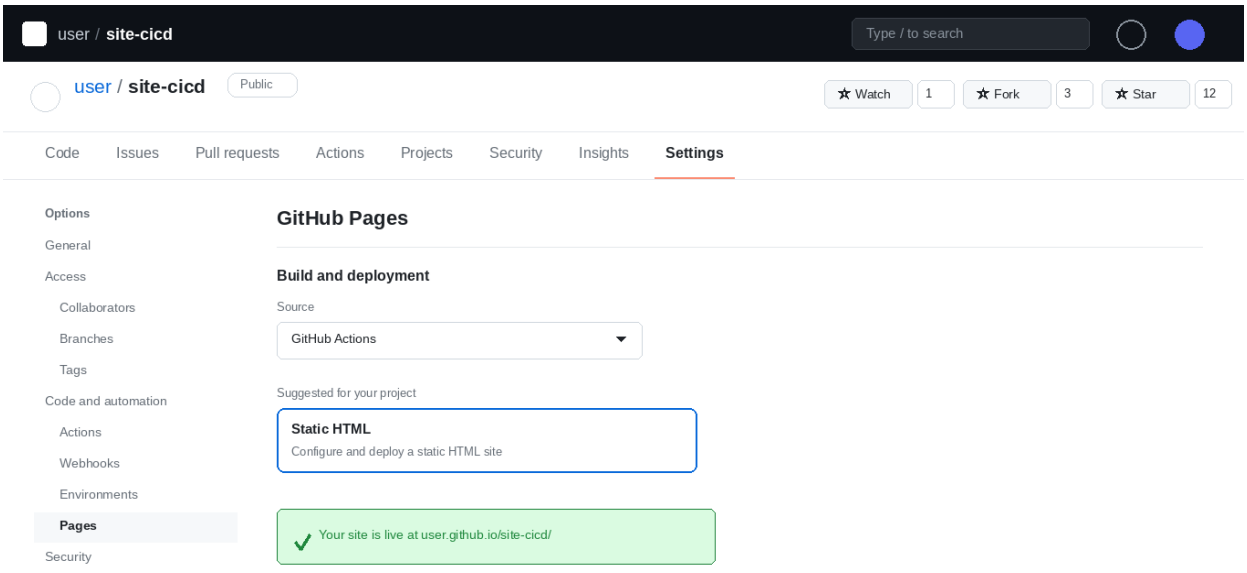


Рисунок 4.3 — Включение GitHub Pages

На этом шаге сайт публикуется автоматически при каждом push в main, но без гибкой настройки процесса сборки — это ограничение встроенного способа, которое снимается настройкой собственного workflow (упражнение 3).

Упражнение 3. Создание workflow-файла GitHub Actions

1. В репозитории создайте каталог .github/workflows и в нём файл deploy.yml (Add file → Create new file, в поле имени введите .github/workflows/deploy.yml — GitHub сам создаст вложенные папки).

2. Внесите в файл следующее содержимое:

```
name: Deploy static site

on:
  push:
    branches: [ main ]

permissions:
  contents: read
  pages: write
  id-token: write

jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Pages
        uses: actions/configure-pages@v5

      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: '.'

      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

3. Опишите коммит, например «Добавлен workflow автодеплоя», и нажмите Commit changes.

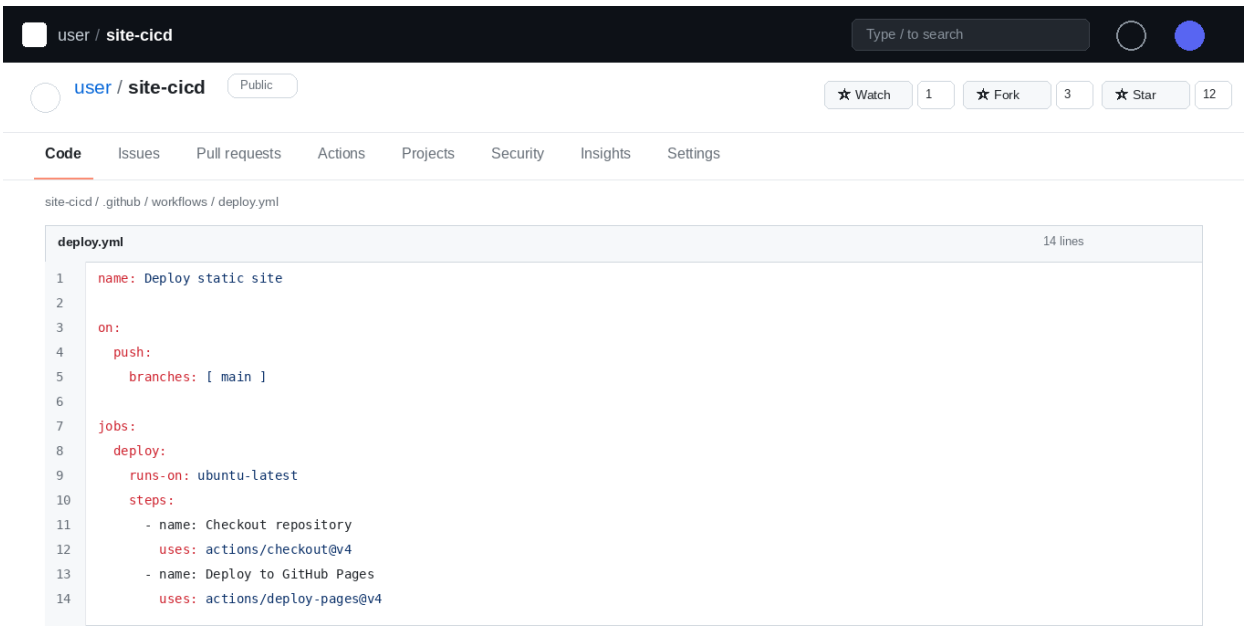


Рисунок 4.4 — Workflow-файл в репозитории

4. Перейдите на вкладку Actions репозитория и убедитесь, что запустился workflow Deploy static site. Дождитесь появления зелёной галочки, означающей успешное выполнение всех шагов.

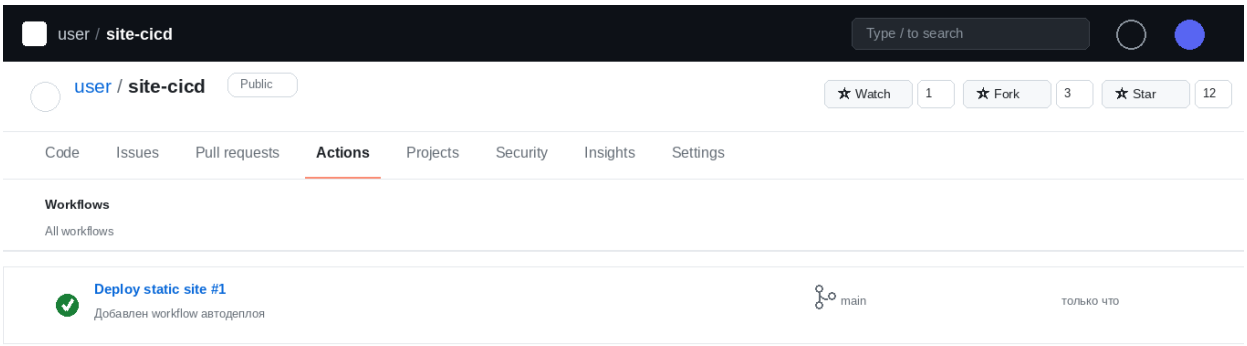


Рисунок 4.5 — Успешный запуск workflow

Упражнение 4. Проверка автоматического обновления сайта

1. Откройте файл index.html, измените текст заголовка (например, добавьте фразу «Обновлено через CI/CD») и сохраните изменение прямо на GitHub.

2. Сразу после сохранения на вкладке Actions должен автоматически запуститься новый workflow — без каких-либо дополнительных действий пользователя.

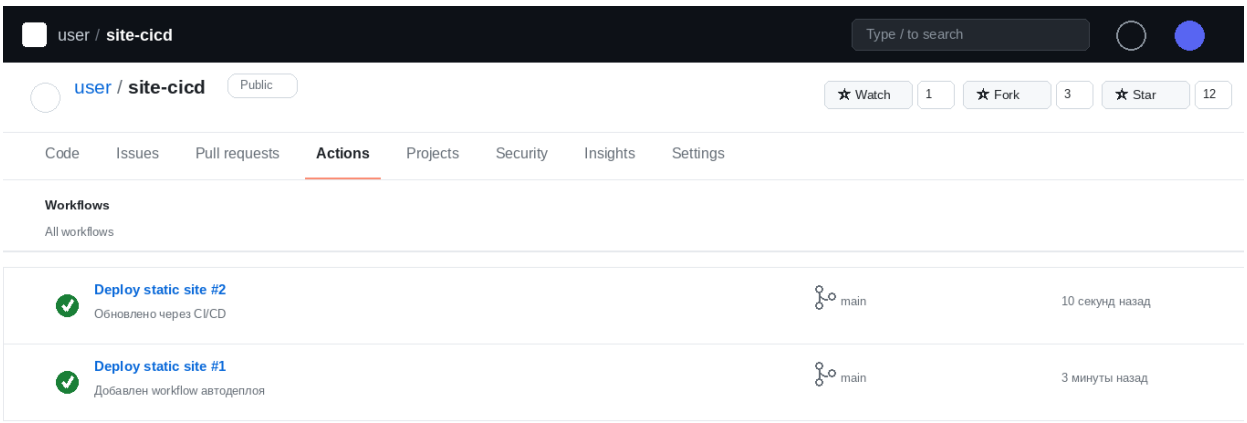


Рисунок 4.6 — Автозапуск workflow после изменения файла

3. После завершения workflow откройте ссылку сайта и обновите страницу (Ctrl+F5). Убедитесь, что изменения опубликованы.

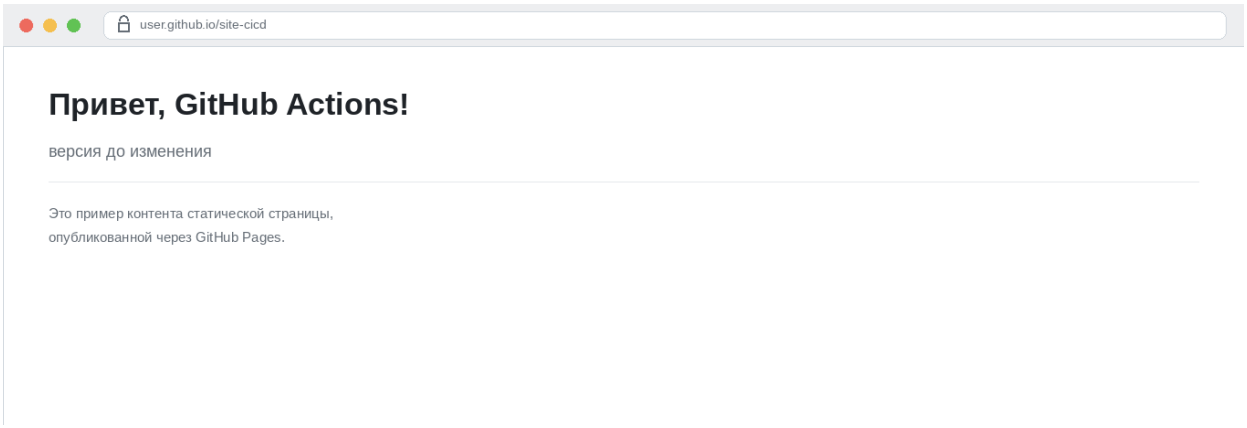


Рисунок 4.7 — Сайт до изменения

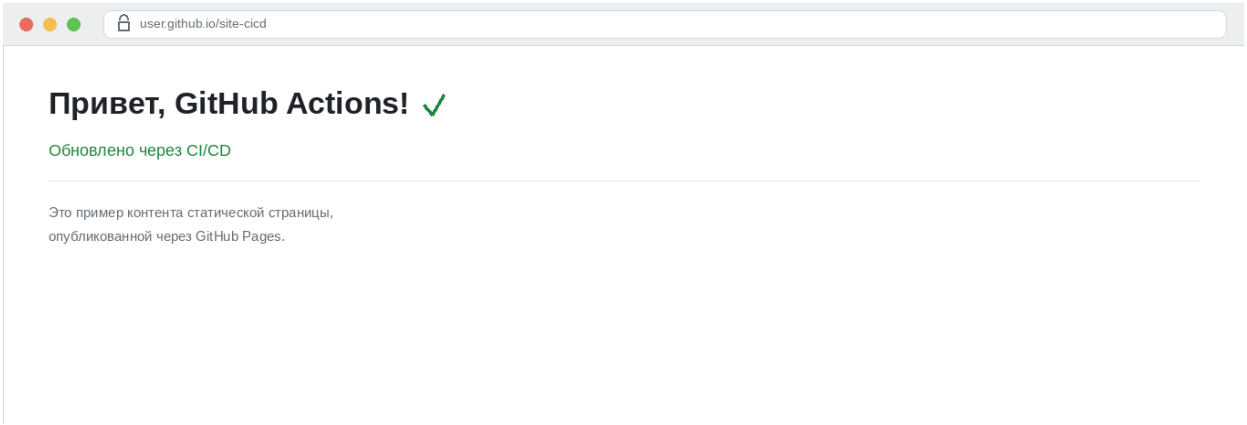


Рисунок 4.8 — Сайт после автоматического обновления

Упражнение 5. Альтернативный способ: нативный автодеплой на Vercel

1. Зарегистрируйтесь на https://vercel.com/ и авторизуйтесь через свой аккаунт GitHub.

2. Нажмите Add New → Project, выберите репозиторий site-cicd из списка и нажмите Import.

3. Оставьте настройки сборки по умолчанию (для статического сайта framework preset — Other) и нажмите Deploy.

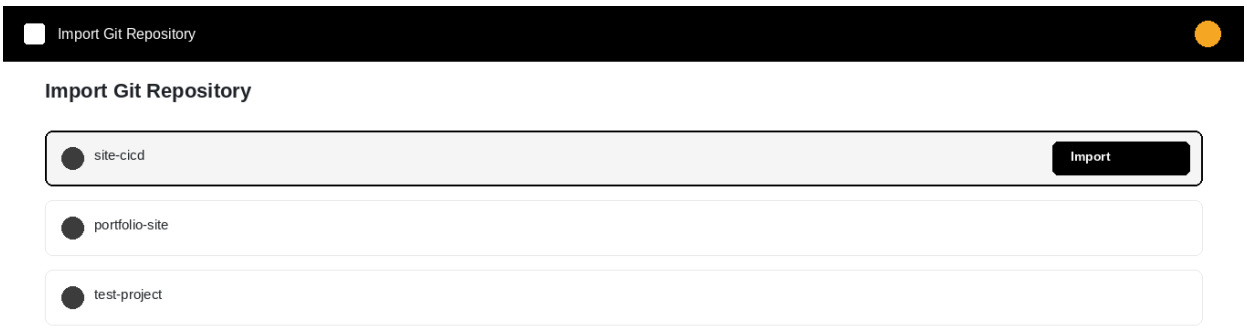


Рисунок 4.9 — Подключение репозитория к Vercel

4. Внесите изменение в файл репозитория на GitHub и убедитесь, что Vercel самостоятельно, без отдельного workflow-файла, запускает новую сборку и публикует изменения — интеграция уже включена на уровне платформы.



Рисунок 4.10 — Автоматический деплой в Vercel

5. Сравните оба способа (GitHub Actions → GitHub Pages и нативная интеграция Vercel) и отразите наблюдения в выводе отчёта.

Упражнение 6. Индивидуальный проект

Выполните собственный проект с применением настроенного CI/CD-пайплайна. Примеры тем:

- Доработка workflow: добавление шага проверки HTML/CSS (линтинг) перед деплоем.
- Настройка деплоя нескольких страниц с общим шаблоном (например, простое портфолио).
- Добавление статус-бейджа workflow в файл README.md, отображающего текущее состояние сборки.
- Настройка деплоя проекта на сборке (например, простой статический генератор) вместо чистого HTML.

Тему индивидуального проекта согласуйте с преподавателем до начала выполнения.

5. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. Документация GitHub Actions. URL: https://docs.github.com/actions (дата обращения: 2026).

2. Документация GitHub Pages. URL: https://docs.github.com/pages (дата обращения: 2026).

3. Документация Vercel. URL: https://vercel.com/docs (дата обращения: 2026).

4. Программная инженерия. Интеграционный подход к разработке / Е. Л. Турнецкая, А. В. Аграновский. — Санкт-Петербург: Лань, 2023. — 216 с. — ISBN 978-5-507-46898-0.

ПРИЛОЖЕНИЕ А

ТИТУЛЬНЫЙ ЛИСТ ОТЧЁТА

Бланк титульного листа отчёта по лабораторной работе скачайте по ссылке

https://guap.ru/c/regdocs/docs/uch

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего образования

«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА №41

ОТЧЁТ

ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

_____ должность, уч. степень, звание подпись, дата инициалы, фамилия

ЛАБОРАТОРНАЯ РАБОТА №3
«АВТОМАТИЗАЦИЯ РАЗВЁРТЫВАНИЯ ПРОЕКТА С ПОМОЩЬЮ GITHUB ACTIONS (CI/CD)»

по дисциплине: ИНФОРМАТИКА

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. № _____ подпись, дата инициалы, фамилия

Санкт-Петербург 2026

ПРИЛОЖЕНИЕ Б

ПРИМЕР ОТЧЁТА ПРИ ВЫПОЛНЕНИИ РАБОТЫ

Цель и задачи практической работы

Цель: получение практических навыков настройки автоматического развёртывания (CI/CD) веб-проекта на хостинге с использованием сервиса GitHub Actions.

Задачи: создать репозиторий проекта; опубликовать сайт на хостинге; настроить workflow автоматического деплоя; проверить автообновление сайта; выполнить индивидуальный проект.

Выполнение практической работы

1. Создание репозитория. Репозиторий portfolio-site создан на GitHub, в него добавлены файлы index.html и style.css.

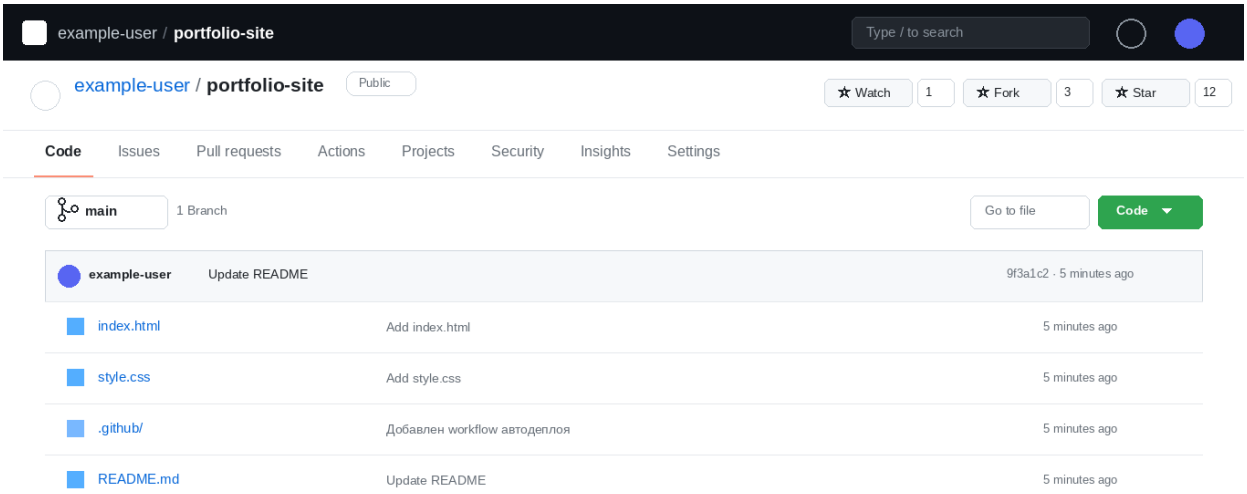


Рисунок 1 — Репозиторий проекта

2. Настройка workflow. Создан файл .github/workflows/deploy.yml, использующий actions/checkout, actions/configure-pages, actions/upload-pages-artifact и actions/deploy-pages для публикации сайта на GitHub Pages.

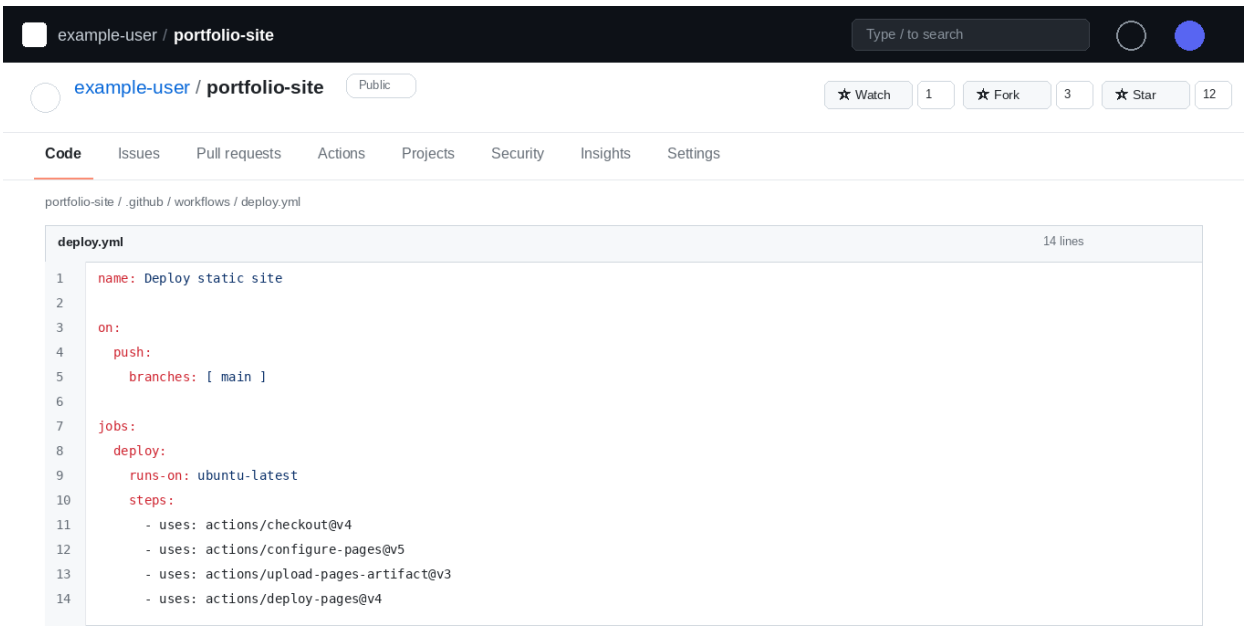


Рисунок 2 — Workflow-файл

3. Проверка автозапуска. После commit изменений во вкладке Actions отображается успешно завершённый запуск workflow Deploy static site.

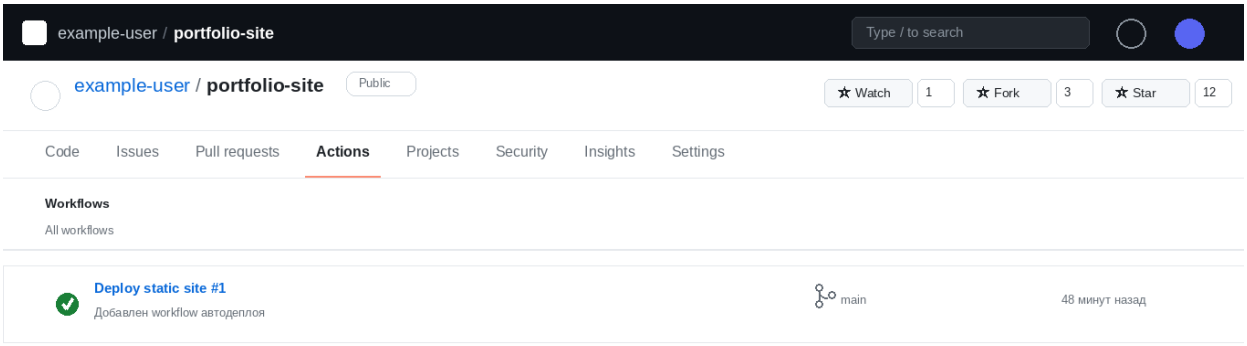


Рисунок 3 — Результат выполнения workflow

4. Проверка автообновления. После изменения заголовка страницы и push в main сайт на GitHub Pages обновился автоматически в течение минуты, без ручной публикации.

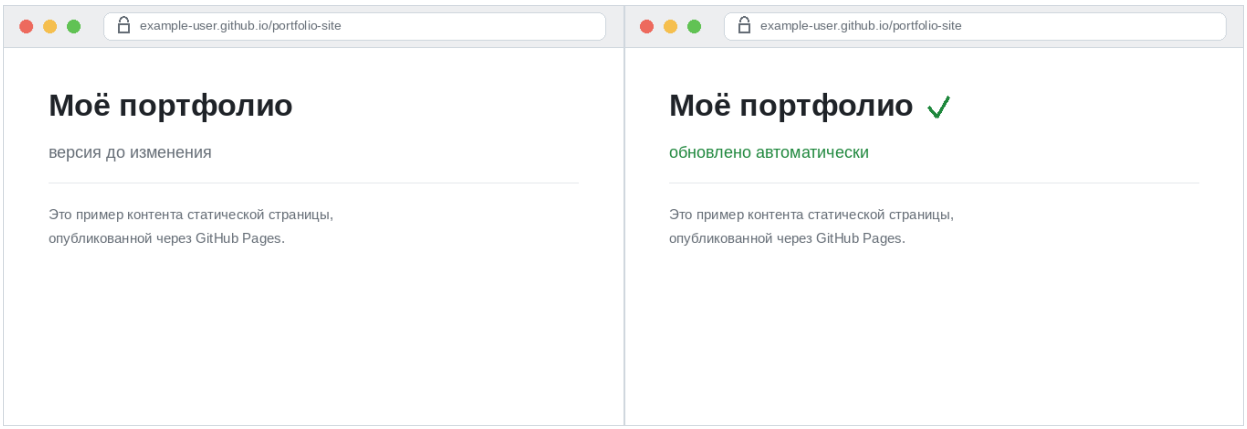


Рисунок 4 — Сайт до и после автообновления

5. Ссылка на репозиторий: https://github.com/example-user/portfolio-site

Ссылка на опубликованный сайт: https://example-user.github.io/portfolio-site/

Индивидуальный проект

В рамках индивидуального проекта в workflow добавлен статус-бейдж в README.md, отображающий текущее состояние последней сборки, а также шаг проверки корректности HTML-разметки перед деплоем.

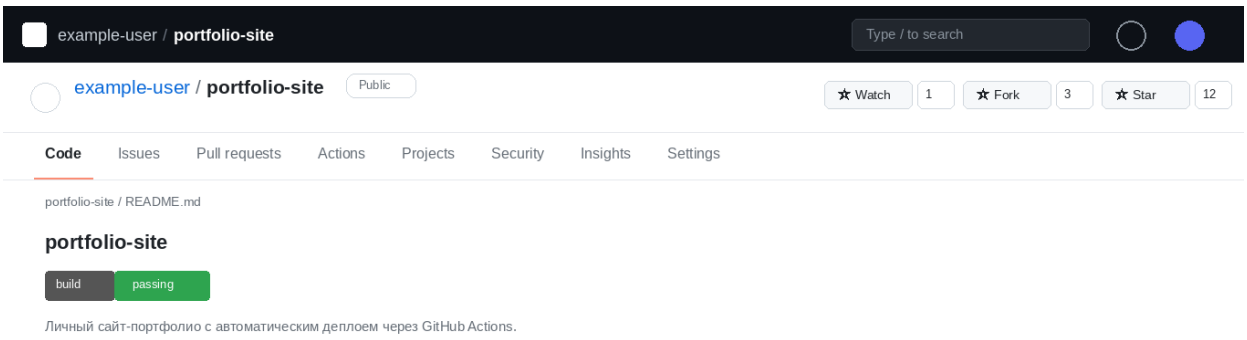


Рисунок 5 — Статус-бейдж в README

Вывод

Целью данной практической работы было освоение навыков настройки автоматического развёртывания веб-проекта с использованием GitHub Actions. В ходе работы был создан репозиторий со статическим сайтом, настроен workflow-файл, автоматически запускающий сборку и публикацию проекта при каждом push, а также проверена работа механизма на практике: изменение файла в репозитории приводило к автоматическому обновлению сайта на хостинге без ручных действий.

Наибольшую сложность вызвала настройка прав доступа workflow к разделу Pages (permissions: pages: write) — без явного указания этих прав первый запуск завершался ошибкой. Также потребовалось время на понимание разницы между встроенной публикацией из ветки и явным workflow: второй способ оказался более гибким, так как позволяет добавлять произвольные шаги обработки перед публикацией.

Сравнение с нативной интеграцией Vercel показало, что оба подхода решают одну задачу разными средствами: GitHub Actions даёт полный контроль над процессом сборки, а готовые интеграции хостингов проще в первоначальной настройке, но менее прозрачны. Полученные навыки настройки CI/CD являются базовыми для дальнейшей работы с более сложными пайплайнами непрерывной интеграции.

Список использованных источников

1. Документация GitHub Actions. URL: https://docs.github.com/actions (дата обращения: 2026).

2. Документация GitHub Pages. URL: https://docs.github.com/pages (дата обращения: 2026).

3. Документация Vercel. URL: https://vercel.com/docs (дата обращения: 2026).